Stack Implementation

1. Introduction

A stack is a linear data structure that follows the **Last In First Out (LIFO)** principle. This means that the element that is inserted last into the stack will be the first element to be removed. A simple real-life example of a stack is a pile of plates. When plates are stacked one above another, the plate placed at the top is removed first.

Stacks are widely used in computer science for various applications such as expression evaluation, undo and redo operations in software applications, and memory management. The stack allows insertion and deletion of elements only from one end, called the **top** of the stack.

The main operations performed in a stack are **push, pop, and peek**. The push operation inserts an element into the stack, the pop operation removes the top element, and the peek operation displays the top element without removing it.

---

2. Steps / Algorithms

Algorithm for Push

Step 1: Start
Step 2: Check if the stack is full
Step 3: If full, display **Stack Overflow**
Step 4: Otherwise increase the value of top
Step 5: Insert the new element into the stack
Step 6: Stop

Algorithm for Pop

Step 1: Start
Step 2: Check if the stack is empty
Step 3: If empty, display **Stack Underflow**
Step 4: Otherwise remove the top element
Step 5: Decrease the value of top
Step 6: Stop

Algorithm for Peek

Step 1: Start
Step 2: Check if the stack is empty
Step 3: If empty, display **Stack is Empty**
Step 4: Otherwise display the top element
Step 5: Stop

---

3. Code (Java Program)

```
import java.util.Scanner;

public class StackDemo {

  int stack[] = new int[5];
  int top = -1;

  void push(int value) {
     if (top == 4) {
        System.out.println("Stack Overflow");
     } else {
        top++;
        stack[top] = value;
        System.out.println("Inserted: " + value);
     }
```

```
    }

    void pop() {
        if (top == -1) {
            System.out.println("Stack Underflow");
        } else {
            System.out.println("Deleted: " + stack[top]);
            top--;
        }
    }

    void display() {
        if (top == -1) {
            System.out.println("Stack is Empty");
        } else {
            System.out.println("Stack Elements:");
            for (int i = top; i >= 0; i--) {
                System.out.println(stack[i]);
            }
        }
    }

    public static void main(String[] args) {
        StackDemo s = new StackDemo();
        s.push(10);
        s.push(20);
        s.push(30);
        s.display();
        s.pop();
        s.display();
    }
}
```

---

4. Debugging Method

Debugging is the process of identifying and fixing errors in a program. In stack implementation, the most common errors are **stack overflow** and **stack underflow**. Stack overflow occurs when the program tries to insert an element into a full stack. Stack underflow occurs when the program tries to remove an element from an empty stack.

To avoid these errors, the program must always check whether the stack is full before performing the push operation and check whether the stack is empty before performing the pop operation. By handling these conditions properly, the stack program can work correctly without runtime errors.

---

5. Output

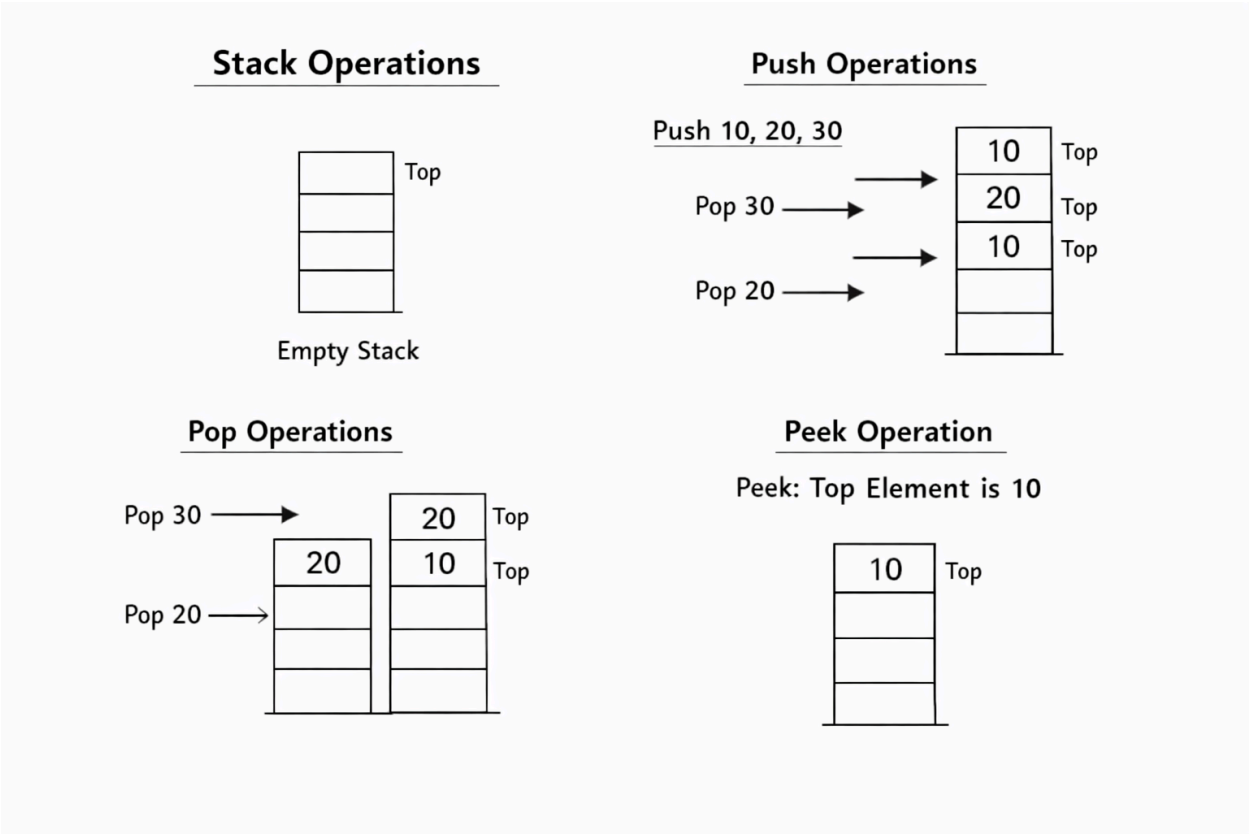
Example Output:

```
Inserted: 10
Inserted: 20
Inserted: 30

Stack Elements:
30
20
10

Deleted: 30

Stack Elements:
20
10
```


Stack Operations
Top
Empty Stack
Push Operations
Push 10, 20, 30
Pop 30
Pop 20
10
20
10
Top
Top
Top
Pop Operations
Pop 30
Pop 20
20
20
10
Top
Top
Peek Operation
Peek: Top Element is 10
10
Top